QueryAPIGate BLOG · BENCHMARK

Every run, recorded

The Rust data plane served calls faster than QueryAPIGate could record them: at the peak, half the runs never reached the audit trail. An optional ClickHouse run store records every one, in about 13 bytes, with reads in milliseconds at 250 million runs. Experimental, measured, reviewed.

Published 29 October 2026 · Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?p=every-run-recorded

---

QueryAPIGate is a governed gateway: a saved query becomes `/q/<name>`, and every call is checked, run, masked where it has to be, and **recorded**. Who called, which query, when, how many rows, what was refused. That record is run history, the event log behind `/events`, and the usage reports. For a gateway that sits in front of production data, it's not a nice-to-have. It's half the point.

In 0.44 we put a Rust data plane in front of the Python server: Python decides, Rust serves. It served about 8 times the calls a second per core. And it found the next limit straight away. The data plane reported every run back to Python, and Python wrote each one twice to the metadata store, SQLite or PostgreSQL: one row in history, one in the event log. At the 64-caller peak, the data plane served 34,240 calls a second and **52% of the runs were recorded**. The rest were counted as dropped.

The gateway kept up. Its audit trail didn't.

Why ClickHouse

At 20,000 calls a second, every run recorded is 1.7 billion events a day. That's more than SQLite or the PostgreSQL store should hold, and the metadata store has other work to do: keys, queries, connections, the audit log. What runs need is different. They're written once and never changed, read mostly by time, counted in aggregates, and dropped when they age out.

That's what ClickHouse is built for. It compresses columns of similar values very well, takes large batched inserts cheaply, and can drop a whole partition when its time is up instead of deleting rows. So 0.45 adds an optional **run store**: set `QUERYAPIGATE_RUN_STORE_URL`, and run history, the event log and usage rollups are kept in ClickHouse. Everything that decides who may do what stays in the metadata store.

How it's built

One insert per batch. A batch of runs is one insert into a table called `events`: up to 100,000 runs from the data plane. ClickHouse fills history, per-query run counts and per-minute and per-hour usage rollups from that one insert, through materialized views. Each table's retention is a TTL on its partitions.

The data plane writes its own runs. It gets its own ClickHouse user that can only insert into `events`. It can't read anything. The views that fill the other tables run as the user who created them. Python is told only how many runs there were, for `/metrics`.

Event ids that clients can trust. `/events` clients resume with `Last-Event-ID`, so ids must keep increasing and must never be handed out twice. Two kinds of writer now mint them, Python and the data plane, with no shared counter. So an id is the millisecond since 2026-01-01, times 4,096, plus a number from a block of each millisecond that the writer leases in the metadata store. Two writers never hold the same block. Ids stay below 2^53, so a JavaScript client holds them exactly.

Delivered once settled. An insert that's acknowledged later than another can still carry the lower id. So the events server delivers an event only once it's a second old, and a client resuming from any id it was given misses nothing. A rare insert later than that is counted in `/metrics`.

What the first measurement found

We measured it the way we measured the data plane: on a fixed laptop, against the store it replaces, with the go or no-go criteria written down first (ADR 0029, section 9). It didn't pass at first.

- **The data plane's batches were too small.** At 500 runs an insert, ClickHouse spent 1.6 cores merging parts. At up to 100,000, it spent 0.20.
- **The usage report took 4,991 ms.** It parsed every run's JSON entry to find its error code, masking and statement. Those became columns written with the run, then whole minutes came from the per-minute rollup: 16 ms, and still exact.
- **The Console's version list took 723 ms.** It scanned a query's whole history for its run counts. A small table, kept up by a view, now holds them: 6 ms.
- **A filtered replay of the log took 41 seconds.** It read the large `entry` column for nearly every block to find 1 match in 2,400. It now finds a page's ids first, then reads only those rows.

Each one came from ClickHouse's own query log, not from guessing.

The numbers

	0.44'S STORE	THE RUN STORE
Runs recorded, 20,000 calls a second for 30 minutes	-	35,999,897 of 35,999,897
Runs recorded, 64-caller peak	52-54%	100%
Gateway cores at 20,000 calls a second	3.40	3.02 (-11%)
Of which Python	0.49	0.13
A run, compressed	-	about 13 bytes

Clients that hung up every 3 to 10 seconds and resumed at random got 35 million events over 7,515 reconnects: none missing, none repeated, none out of order. That was the result we cared about most.

A security review, then 250 million runs

A new store that holds every run's record needed a review before it could graduate. It found six things. The three that mattered:

- **A forged row could split an event stream.** The events server copied a stored event into its frame as it was. A line break in a row, written by anyone with an insert credential, could start frames of the writer's own, with an id that moved every client's resume point. Frames are one line now, and the table refuses ids above 2^53.
- **A data plane pointed at another ClickHouse would have written its runs there,** out of history and out of reach of an erasure. Python's instances already refused to start in that case; the data plane didn't check. Now Python tells it where runs belong, and it writes nothing until they agree.
- **Ad-hoc SQL text lived longer than the retention settings said,** in the hourly rollups, for 400 days. They keep a hash of each statement now.

Then the measurement the laptop's disk hadn't allowed the first time: 250 million runs, from a 30-minute soak's real runs copied through the same views.

AT 250 MILLION RUNS	RESULT	THE BAR
A run in the event log, compressed	13.8 bytes	40
The newest page of history, p95	12.5 ms	100 ms
One key's newest page, p95	73.6 ms	100 ms
A 24-hour usage report, p95	66.7 ms	500 ms
One key's replay of the whole log	11.8 s, then 6.2 s	10 s

134 GB of raw entries became 6.9 GB. The replay missed its bar. Each page of 1,000 ids was its own read, and at this size a read costs 0.1–0.2 seconds of planning whatever it finds, where all 35,000 of the key's ids come back in one read in half a second. A replay now finds up to 100,000 ids at a time. On the same data, 12.0 seconds became 6.2.

Where it doesn't help, and what's left

- **It's experimental.** It shipped in 0.45.0. The review's fixes and the faster replay are in the next release, and it graduates once a release goes out unchanged.
- **The data plane writing its own runs needs the data plane,** which is still built from the repository, not shipped in the image.
- **One laptop, one ClickHouse server.** On a replicated cluster, a read from a replica behind the one that took the insert could miss the settle window. That's an open question.

- **The events server is the next ceiling.** One `queryapigate events` process delivers about 6,500 to 10,000 events a second per stream. Run several behind a load balancer, each reads the store on its own.
- **Live events arrive about a second later,** by design, and history is kept by time, 30 days unless you set it.

If you run the data plane, or record enough runs that the metadata store struggles, guide 65 switches it on. Try it on a test deployment first. All the figures are in the first measurement and the 250-million-run one, with the harness to run them yourself.

READ NEXT
Python decides, Rust serves

QUESTIONS OR PROBLEMS
Discord · GitHub issues

LATEST VERSION OF THIS PAGE
anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?p=every-run-recorded

FOLLOW QUERYAPIGATE
GitHub X Discord Bluesky Mastodon YouTube Reddit
@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate